Illustration of EIF Estimator

Luke Benz

2026-05-01

Introduction

Given that the underlying EHR data can not be shared, the example presented in the main text can not be reproduced locally. For a detailed reproducible example, we have created a working example based on OMOP-CDM formatted data. In this file, we walk through implementation of the estimator as well as some diagnostic checks, figures, and commentary.

```
### Load Packages and Helper Functions
library(tidyverse)
library(SuperLearner)
library(here)
library(ggridges)
source(here('scripts/helpers.R'))
```

We begin by loading in the synthetic data example created in `build_omop_example.R` using the `{omock}` package. The data contain information on 10,677 surgical patients undergoing Roux-en-Y gastric bypass ($A = 1$) or sleeve gastrectomy $A = 0$.

- A = surgical procedure
- Y = % change in BMI from baseline after 3 years
- $\boldsymbol{L}^*$: race (1 = Caucasian, 0 = Non-Caucasian), sex (1 = Female, 0 = Male), Age
- $\boldsymbol{L}_m^e$: Baseline A1c and Baseline BMI
- E: Eligibility indicator for BMI ≥ 35 kg/m^2 and A1c $\geq 6.5\%$
- R: Indicator for eligibility ascertainment.

When creating this synthetic dataset, we created outcomes for all patients and masked study eligibility for some patients. In practice, patients may be missing outcomes as well, in which case imputation or switching to $\widehat{\theta}_{\text{IF}}$ are both options. For this example, rather than demonstrate $\widehat{\theta}_{\text{EIF}}$ over a range of lookback windows, we simply take the baseline values to be the most recent measures within the year prior to surgery. Implementation of $\widehat{\theta}_{\text{EIF}}$ would be identical after changing the operationalization of study eligibility. Finally, we note that these data are generated such that the true value $\theta_{\text{ATT}}^{\text{elig}} = -0.06$

```
### Load in Data
df_analysis <-
  read_csv(here('scripts/worked_omop_example/analysis_dataset.csv')) %>%
  rename('A' = bs_type)


### Summary S
df_analysis %>%
```

```
  group_by(eligible, R) %>%
  count() %>%
  ungroup() %>%
  mutate('pct' = n/sum(n))
```

```
## # A tibble: 3 x 4
##   eligible     R     n   pct
##      <dbl> <dbl> <int> <dbl>
## 1        0     1  4195 0.393
## 2        1     1  1822 0.171
## 3       NA     0  4660 0.436
```

We see that about 44% of patients have insufficient information to determine study eligibility.

Implementing $\widehat{\theta}_{\mathbf{EIF}}$

First, we present a function for computing θ_{EIF} below, which takes the following arguments

- `df`: data frame containing treatment variable (must be called A), eligibility (must be called eligible), ascertainment (must be called R), covariates, and outcome
- `models`: list of specifications for each nuisance model
- `n_splits` = # of splits to be used in sample splitting

```
### eif_estimator: function to compute EIF estimator
### Arguments
###   df: data frame containing treatment variable (must be called A),
###       eligibility (must be called eligible), ascertainment (must be called R), covariates, and outc
###   models: list of specifications for each nuisance model.
###   n_splits = # of splits to be used in sample splitting.
###
### Output: List w/ Data frame with treatment effect and standard error and individual IF contributions,

eif_estimator <- function(df, models, n_splits) {
  ### Unpack nuisance model instructions
  eta <- models$eta
  epsilon <- models$epsilon
  mu <- models$mu
  xi <- models$xi
  u <- models$u
  gamma <- models$gamma
  chi <- models$chi

  ### Create storage for predictions
  df <-
    df %>%
    mutate('eta1_hat' = NA,
           'eta0_hat' = NA,
           'epsilon1_hat' = NA,
           'mu0_hat' = NA,
           'xi_hat' = NA,
           'u_hat' = NA,
```

```r
          'gamma_hat' = NA,
          'chi_hat' = NA)


### Sample split
split_ids <-
  split(sample( 1:nrow(df) ), ceiling(1:nrow(df) / ceiling(nrow(df)/n_splits)))

for(i in 1:n_splits) {
  test_ids <- split_ids[[i]]
  train_ids <- setdiff(1:nrow(df), test_ids)
  df_train <- dplyr::slice(df, train_ids)
  df_test <- dplyr::slice(df, test_ids)

  ### Fit eta
  cat('Fitting ETA on Split [', i, '/', n_splits, ']\n', sep = '')
  if(eta$type == 'GLM') {
    if(!eta$stratify) {
      eta_fit <- glm(eta$formula, family = 'binomial', data = df_train)
      eta1_hat <- predict(eta_fit, newdata = mutate(df_test, 'A' = 1), type = 'response')
      eta0_hat <- predict(eta_fit, newdata = mutate(df_test, 'A' = 0), type = 'response')
    } else {
      eta1_fit <- glm(eta$formula, family = 'binomial', data = filter(df_train, A == 1))
      eta1_hat <- predict(eta1_fit, newdata = mutate(df_test, 'A' = 1), type = 'response')
      eta0_fit <- glm(eta$formula, family = 'binomial', data = filter(df_train, A == 0))
      eta0_hat <- predict(eta0_fit, newdata = mutate(df_test, 'A' = 0), type = 'response')
    }
  } else if(eta$type == 'super_learner') {

    if(!eta$stratify) {
      ### Create Datasets for Super Learner
      Y_train <-
        df_train %>%
        pull(!!eta$Y)

      X_train <-
        df_train %>%
        select(all_of(!!eta$X))

      X_test <-
        df_test %>%
        select(all_of(!!eta$X))

      ### Convert to Matrix then back to DF to handle cases where we specify interactions
      X_train_mat <- model.matrix(eta$formula, X_train)[,-1] ### Rm intercept term since some other m
      colnames(X_train_mat) <- gsub(':', '_', colnames(X_train_mat))

      X_test_mat_0 <- clean_test_matrix(model.matrix(eta$formula, mutate(X_test, 'A' = 0))[,-1], X_tra
      X_test_mat_1 <- clean_test_matrix(model.matrix(eta$formula, mutate(X_test, 'A' = 1))[,-1], X_tra

      X_train_df <- as.data.frame(X_train_mat)
      X_test_df_0 <- as.data.frame(X_test_mat_0)
      X_test_df_1 <- as.data.frame(X_test_mat_1)
```

```r
    ### Random Forest Hyperparameters
    mtry_seq <- floor(sqrt(ncol(X_train_mat)) * c(0.5, 1))
    tree_seq <- c(250, 500)
    min_n_seq <- c(5, 30)
    rf_learners <-
      create.Learner("SL.ranger", tune = list('mtry' = mtry_seq,
                                              'num.trees' = tree_seq,
                                              'min.node.size' = min_n_seq))
    sl_eta <-
      SuperLearner(Y = Y_train,
                   X = X_train_df,
                   family = 'binomial',
                   SL.library = c(rf_learners$names, eta$sl_libs))

    eta1_hat <- predict.SuperLearner(sl_eta, newdata = X_test_df_1, onlySL = T)$pred
    eta0_hat <- predict.SuperLearner(sl_eta, newdata = X_test_df_0, onlySL = T)$pred
  } else if(eta$stratify) {
    ### Create Datasets for Super Learner
    Y_train_0 <-
      df_train %>%
      filter(A == 0) %>%
      pull(!!eta$Y)

    Y_train_1 <-
      df_train %>%
      filter(A == 1) %>%
      pull(!!eta$Y)

    X_train_0 <-
      df_train %>%
      filter(A == 0) %>%
      select(all_of(!!eta$X))

    X_train_1 <-
      df_train %>%
      filter(A == 1) %>%
      select(all_of(!!eta$X))

    X_test <-
      df_test %>%
      select(all_of(!!eta$X))

    ### Random Forest Hyperparameters
    mtry_seq <- floor(sqrt(ncol(X_train_0)) * c(0.5, 1))
    tree_seq <- c(250, 500)
    min_n_seq <- c(5, 30)
    rf_learners <-
      create.Learner("SL.ranger", tune = list('mtry' = mtry_seq,
                                              'num.trees' = tree_seq,
                                              'min.node.size' = min_n_seq))

    ### Fit each Super Learner Seperately
    sl_eta_0 <-
```

```
      SuperLearner(Y = Y_train_0,
                   X = X_train_0,
                   family = 'binomial',
                   SL.library = c(rf_learners$names, eta$sl_libs))

    sl_eta_1 <-
      SuperLearner(Y = Y_train_1,
                   X = X_train_1,
                   family = 'binomial',
                   SL.library = c(rf_learners$names, eta$sl_libs))


    eta1_hat <- predict.SuperLearner(sl_eta_1, newdata = X_test, onlySL = T)$pred
    eta0_hat <- predict.SuperLearner(sl_eta_0, newdata = X_test, onlySL = T)$pred

  }

}

df$eta1_hat[test_ids] <- eta1_hat
df$eta0_hat[test_ids] <- eta0_hat

### Epsilon Star
cat('Fitting EPSILON on Split [', i, '/', n_splits, ']\n', sep = '')
if(epsilon$type == 'GLM') {
  epsilon_fit <- glm(epsilon$formula, family = 'binomial', data = filter(df_train, R == 1))
  epsilon1_hat <- predict(epsilon_fit, newdata = mutate(df_test, 'A' = 1), type = 'response')
} else if(epsilon$type == 'super_learner') {
  ### Create datasets to train super learner
  Y_train <-
    df_train %>%
    filter(R == 1) %>%
    pull(!!epsilon$Y)

  X_train <-
    df_train %>%
    filter(R == 1) %>%
    select(all_of(epsilon$X))

  ### Set to predict on
  X_test <-
    df_test %>%
    mutate('A' = 1) %>%
    select(all_of(epsilon$X))


  ### Random Forest Hyperparameters
  mtry_seq <- floor(sqrt(ncol(X_train)) * c(0.5, 1))
  tree_seq <- c(250, 500)
  min_n_seq <- c(5, 30)
  rf_learners <-
    create.Learner("SL.ranger", tune = list('mtry' = mtry_seq,
                                            'num.trees' = tree_seq,
```

```
                                                 'min.node.size' = min_n_seq))
    sl_epslion <-
      SuperLearner(Y = Y_train,
                   X = X_train,
                   family = 'binomial',
                   SL.library = c(rf_learners$names, epsilon$sl_libs))


    epsilon1_hat <-
      as.vector(predict.SuperLearner(sl_epslion, newdata = X_test, onlySL = T)$pred)

  }

  df$epsilon1_hat[test_ids] <- epsilon1_hat

  ### Fit Mu
  cat('Fitting MU0 on Split [', i, '/', n_splits, ']\n', sep = '')
  if(mu$type == 'LM') {
    mu_fit <- lm(mu$formula, data = filter(df_train, R == 1, eligible == 1 | !mu$elig_only, A == 0 |
    mu0_hat <- predict(mu_fit, newdata = mutate(df_test, 'A' = 0))
    df_train$mu0_hat <- predict(mu_fit, newdata = mutate(df_train, 'A' = 0)) ### needed for other mod
  } else if(mu$type == 'super_learner') {
    ### Create datasets to train super learner
    Y_train <-
      df_train %>%
      filter(R == 1, A == 0 | !mu$stratify) %>%
      filter(eligible == 1 | !mu$elig_only) %>%
      pull(!!mu$Y)

    X_train <-
      df_train %>%
      filter(R == 1, A == 0 | !mu$stratify) %>%
      filter(eligible == 1 | !mu$elig_only) %>%
      select(all_of(mu$X))

    X_train_mat <- model.matrix(mu$formula, X_train)[,-1] ### Rm intercept term since some other mode
    colnames(X_train_mat) <- gsub(':', '_', colnames(X_train_mat))

    ### Set to predict on
    X_test <-
      df_test %>%
      select(all_of(mu$X)) %>%
      mutate('A' = 0)
    cc_ids <- which(!is.na(X_test$baseline_a1c) & !is.na(X_test$baseline_bmi))
    X_test_mat <- clean_test_matrix(model.matrix(mu$formula, X_test)[,-1], X_train_mat)


    ### Random Forest Hyperparameters
    mtry_seq <- floor(sqrt(ncol(X_train_mat)) * c(0.5, 1))
    tree_seq <- c(250, 500)
    min_n_seq <- c(5, 30)
    rf_learners <-
      create.Learner("SL.ranger", tune = list('mtry' = mtry_seq,
```

```
                                                 'num.trees' = tree_seq,
                                                 'min.node.size' = min_n_seq))

    sl_mu <-
      SuperLearner(Y = Y_train,
                   X = X_train_mat,
                   SL.library = c(rf_learners$names, mu$sl_libs))

    mu0_hat <- rep(NA, nrow(df_test))
    mu0_hat[cc_ids] <- predict.SuperLearner(sl_mu, newdata = X_test_mat, onlySL = T)$pred

    ### needed for Xi and Chi models
    X_train_0 <-
      df_train %>%
      select(all_of(mu$X)) %>%
      mutate('A' = 0)
    cc_ids <- which(!is.na(X_train_0$baseline_a1c) & !is.na(X_train_0$baseline_bmi))
    X_train_mat0 <- clean_test_matrix(model.matrix(mu$formula, X_train_0)[,-1], X_train_mat)
    df_train$mu0_hat[cc_ids] <- predict.SuperLearner(sl_mu, newdata = X_train_mat0, onlySL = T)$pred
  }

  df$mu0_hat[test_ids] <- mu0_hat

  ### Fit U
  cat('Fitting U on Split [', i, '/', n_splits, ']\n', sep = '')
  if(u$type == 'GLM') {
    ### Parametric model for u
    u_fit <- glm(u$formula, family = 'binomial', data = filter(df_train, R == 1, eligible == 1 | !u$el
    u_hat <- predict(u_fit, newdata = df_test, type = 'response')
    df_train$u_hat <- predict(u_fit, newdata = df_train, type = 'response')
  } else if(u$type == 'super_learner') {
    ### Create datasets to train super learner
    Y_train <-
      df_train %>%
      filter(R == 1) %>%
      filter(eligible == 1 | !u$elig_only) %>%
      pull(!!u$Y)

    X_train <-
      df_train %>%
      filter(R == 1) %>%
      filter(eligible == 1 | !u$elig_only) %>%
      select(all_of(u$X))

    ### Set to predict on
    X_test <-
      df_test %>%
      select(all_of(u$X))
    cc_ids <- which(!is.na(X_test$baseline_bmi) & !is.na(X_test$baseline_a1c))

    ### Random Forest Hyperparameters
    mtry_seq <- floor(sqrt(ncol(X_train)) * c(0.5, 1))
    tree_seq <- c(250, 500)
```

```
    min_n_seq <- c(5, 30)
    rf_learners <-
      create.Learner("SL.ranger", tune = list('mtry' = mtry_seq,
                                              'num.trees' = tree_seq,
                                              'min.node.size' = min_n_seq))

    sl_u <-
      SuperLearner(Y = Y_train,
                   X = X_train,
                   family = 'binomial',
                   SL.library = c(rf_learners$names, u$sl_libs))

    u_hat <- rep(NA, nrow(df_test))
    u_hat[cc_ids] <- predict.SuperLearner(sl_u, newdata = X_test[cc_ids,], onlySL = T)$pred
    df$u_hat[test_ids] <- u_hat

    ### needed for Xi and Chi models
    X_train_ <-
      df_train %>%
      select(all_of(u$X))

    cc_ids <- which(!is.na(X_train_$baseline_bmi) & !is.na(X_train_$baseline_a1c))
    df_train$u_hat[cc_ids] <- predict.SuperLearner(sl_u, newdata = X_train_[cc_ids,], onlySL = T)$pre
  }

  ### Create Compound Outcomes for EIF Nuisance Functions
  df_train <-
    df_train %>%
    mutate('Emu0'= eligible * mu0_hat,
           'E_uratio' = eligible * u_hat/(1-u_hat),
           'EY_uratio' = eligible * pct_weight_change * u_hat/(1-u_hat),
           'Emu0_uratio' = eligible * mu0_hat * u_hat/(1-u_hat)) %>%
    mutate('Emu0' = ifelse(eligible == 0, 0, Emu0),
           'E_uratio' = ifelse(eligible == 0, 0, E_uratio),
           'EY_uratio' = ifelse(eligible == 0, 0, EY_uratio),
           'Emu0_uratio' = ifelse(eligible == 0, 0, Emu0_uratio))

  df_test <-
    df_test %>%
    mutate('Emu0'= eligible * mu0_hat,
           'E_uratio' = eligible * u_hat/(1-u_hat),
           'EY_uratio' = eligible * pct_weight_change * u_hat/(1-u_hat),
           'Emu0_uratio' = eligible * mu0_hat * u_hat/(1-u_hat)) %>%
    mutate('Emu0' = ifelse(eligible == 0, 0, Emu0),
           'E_uratio' = ifelse(eligible == 0, 0, E_uratio),
           'EY_uratio' = ifelse(eligible == 0, 0, EY_uratio),
           'Emu0_uratio' = ifelse(eligible == 0, 0, Emu0_uratio))

  cat('Fitting XI on Split [', i, '/', n_splits, ']\n', sep = '')
  if(xi$type == 'super_learner') {
    ### Create datasets to train super learner
    Y_train <-
      df_train %>%
```

```
      filter(R == 1, A == 1) %>%
      pull(!!xi$Y)

    X_train <-
      df_train %>%
      filter(R == 1, A == 1) %>%
      select(all_of(xi$X))

    X_train_mat <- model.matrix(xi$formula, X_train)[,-1] ### Rm intercept term since some other mode
    colnames(X_train_mat) <- gsub(':', '_', colnames(X_train_mat))

    ### Set to predict on
    X_test <-
      df_test %>%
      select(all_of(xi$X))
    X_test_mat <- clean_test_matrix(model.matrix(xi$formula, X_test)[,-1], X_train_mat)


    ### Random Forest Hyperparameters
    mtry_seq <- floor(sqrt(ncol(X_train_mat)) * c(0.5, 1))
    tree_seq <- c(250, 500)
    min_n_seq <- c(5, 30)
    rf_learners <-
      create.Learner("SL.ranger", tune = list('mtry' = mtry_seq,
                                              'num.trees' = tree_seq,
                                              'min.node.size' = min_n_seq))

    sl_xi <-
      SuperLearner(Y = Y_train,
                   X = X_train_mat,
                   SL.library = c(xi$sl_libs, rf_learners$names))

    xi_hat <- predict.SuperLearner(sl_xi, newdata = X_test_mat, onlySL = T)$pred
    df$xi_hat[test_ids] <- xi_hat
  }

  cat('Fitting GAMMA on Split [', i, '/', n_splits, ']\n', sep = '')
  if(gamma$type == 'super_learner') {
    ### Create datasets to train super learner
    Y_train <-
      df_train %>%
      filter(R == 1, A == 0) %>%
      pull(!!gamma$Y)

    X_train <-
      df_train %>%
      filter(R == 1, A == 0) %>%
      select(all_of(gamma$X))

    X_train_mat <- model.matrix(gamma$formula, X_train)[,-1] ### Rm intercept term since some other m
    colnames(X_train_mat) <- gsub(':', '_', colnames(X_train_mat))

    ### Set to predict on
```

```
    X_test <-
      df_test %>%
      select(all_of(gamma$X))
    X_test_mat <- clean_test_matrix(model.matrix(gamma$formula, X_test)[,-1], X_train_mat)


    ### Random Forest Hyperparameters
    mtry_seq <- floor(sqrt(ncol(X_train_mat)) * c(0.5, 1))
    tree_seq <- c(250, 500)
    min_n_seq <- c(5, 30)
    rf_learners <-
      create.Learner("SL.ranger", tune = list('mtry' = mtry_seq,
                                              'num.trees' = tree_seq,
                                              'min.node.size' = min_n_seq))

    sl_gamma <-
      SuperLearner(Y = Y_train,
                   X = X_train_mat,
                   SL.library = c(gamma$sl_libs, rf_learners$names))

    gamma_hat <- predict.SuperLearner(sl_gamma, newdata = X_test_mat, onlySL = T)$pred
    df$gamma_hat[test_ids] <- gamma_hat

  }

  cat('Fitting CHI on Split [', i, '/', n_splits, ']\n', sep = '')
  if(chi$type == 'super_learner') {
    ### Create datasets to train super learner
    Y_train <-
      df_train %>%
      filter(R == 1, A == 0) %>%
      pull(!!chi$Y)

    X_train <-
      df_train %>%
      filter(R == 1, A == 0) %>%
      select(all_of(chi$X))

    X_train_mat <- model.matrix(chi$formula, X_train)[,-1] ### Rm intercept term since some other mod
    colnames(X_train_mat) <- gsub(':', '_', colnames(X_train_mat))

    ### Set to predict on
    X_test <-
      df_test %>%
      select(all_of(chi$X))
    X_test_mat <- clean_test_matrix(model.matrix(chi$formula, X_test)[,-1], X_train_mat)


    ### Random Forest Hyperparameters
    mtry_seq <- floor(sqrt(ncol(X_train_mat)) * c(0.5, 1))
    tree_seq <- c(250, 500)
    min_n_seq <- c(5, 30)
    rf_learners <-
```

```
        create.Learner("SL.ranger", tune = list('mtry' = mtry_seq,
                                                'num.trees' = tree_seq,
                                                'min.node.size' = min_n_seq))

      sl_chi <-
        SuperLearner(Y = Y_train,
                     X = X_train_mat,
                     SL.library = c(chi$sl_libs, rf_learners$names))

      chi_hat <- predict.SuperLearner(sl_chi, newdata = X_test_mat, onlySL = T)$pred
      df$chi_hat[test_ids] <- chi_hat

    }
  }

  ### Compute estimator
  df_final <-
    df %>%
    ### need some value to get canceled by 0 that's not NA
    mutate('eligible' = replace(eligible, R == 0, 0),
           'mu0_hat' = replace(mu0_hat, R == 0, 0),
           'mu0_hat' = replace(mu0_hat, R == 1 & is.na(mu0_hat), 0),
           'u_hat' = replace(u_hat, R == 0, 0),
           'u_hat' = replace(u_hat, R == 1 & is.na(u_hat), 0)
    ) %>%
    ### Compute numerator and denominator one step estimators (eg., uncentered influence functions)
    mutate('alpha_OS' = A * (1 - R/eta1_hat) * epsilon1_hat + A * R * eligible/eta1_hat,
           'beta_OS' =
             A * (epsilon1_hat * pct_weight_change - xi_hat)
           + A * R/eta1_hat * ((eligible - epsilon1_hat) * pct_weight_change - (eligible * mu0_hat - xi_
           - (1-A) * R/eta1_hat * (eligible * u_hat/(1-u_hat) * (pct_weight_change - mu0_hat) - (gamma_
           - (1-A) * eta0_hat/eta1_hat * ((gamma_hat * pct_weight_change - chi_hat))
    ) %>%
    ### Subject Specific Contributions (centered)
    mutate('alpha_IF' = alpha_OS - mean(alpha_OS),
           'beta_IF' = beta_OS - mean(beta_OS)) %>%

    ### Overall ATT IF Subject Contribution (eg. following Takatsu 2023 Lemma S1 or Kennedy 2022 Review
    mutate('subject_IF'= beta_IF/mean(alpha_OS) - mean(beta_OS)/mean(alpha_OS)^2 * alpha_IF)

  ### Estimates of ATT and Variance
  tau_hat <-
    df_final %>%
    summarise('att_hat' = mean(beta_OS)/mean(alpha_OS),
              'sd' = sd(subject_IF)/sqrt(nrow(df)))

  return(list('tau_hat' = tau_hat,
              'df_EIF' = df_final))
}
```

Next, we specify modeling instructions for each nuisance function. Each specification should contain the following structure

- `type` = model type. We strongly recommend super_learner. For select nuisance models parametric

options also exist

- `Y` = modeling target
- `X` = covariates used in model
- `formula` = model specification formula object
- `sl_libs` = libraries to supply to super learner (in addition to combinations of SL.ranger)

```
### Model Instructions for each nuisance function
### Each specification should contain the following structure
###       type = model_type. We strongly recommend super_learner. For select nuisance models parametric
###       Y = modeling target
###       X = covariates used in model
###       formula = model specification formula object
###       sl_libs = libraries to supply to super learner (in addition to combinations of SL.ranger)
###
###       Additional options exist for stratification by A (mu,eta) or fit among eligible population (m
##
model_list <-
  list(
    ### R ~ Lc + A
    'eta' = list('type' = 'super_learner',
                 'stratify' = F,
                 'Y' = 'R',
                 'X' = c('race_source_value', 'gender_source_value', 'baseline_age', 'A'),
                 'formula' = ~race_source_value + gender_source_value + baseline_age + A,
                 'sl_libs' = c('SL.glm', 'SL.gam')),

    ### E ~ Lc + A + Y | R = 1
    'epsilon' = list('type' = 'super_learner',
                     'Y' = 'eligible',
                     'X' = c('race_source_value', 'gender_source_value', 'baseline_age', 'A', 'pct_weig
                     'formula' = ~race_source_value + gender_source_value + baseline_age + A + pct_weig
                     'sl_libs' = c('SL.glm', 'SL.gam')),

    ### Y ~ Lc + Le + A  | R = 1 (usually fit amongst E = 1 as well)
    'mu' = list('type' = 'super_learner',
                'stratify' = F,
                'elig_only' = T,
                'Y' = 'pct_weight_change',
                'X' = c('race_source_value', 'gender_source_value', 'baseline_age', 'baseline_bmi', 'ba
                'formula' = ~race_source_value + gender_source_value + baseline_age + baseline_bmi + ba
                'sl_libs' = c('SL.polymars', 'SL.lm')),

    ### A ~ Lc + Le | R = 1
    'u' = list('type' = 'super_learner',
               'elig_only' = T,
               'Y' = 'A',
               'X' = c('race_source_value', 'gender_source_value', 'baseline_age', 'baseline_bmi', 'bas
               'formula' = ~race_source_value + gender_source_value + baseline_age + baseline_bmi + bas
               'sl_libs' = c('SL.glm', 'SL.gam')),

    ### Emu0 ~ Lc, Y, A = 1, R = 1
    'xi' = list('type' = 'super_learner',
                'Y' = 'Emu0',
                'X' = c('race_source_value', 'gender_source_value', 'baseline_age', 'pct_weight_change'
```

```
                'formula' = ~race_source_value + gender_source_value + baseline_age + pct_weight_change
                'sl_libs' = c('SL.polymars', 'SL.lm')),

    ### E * u/(1-u) ~ Lc, A = 0, R = 1, Y
    'gamma' =  list('type' = 'super_learner',
                    'Y' = 'E_uratio',
                    'X' = c('race_source_value', 'gender_source_value', 'baseline_age', 'pct_weight_cha
                    'formula' = ~race_source_value + gender_source_value + baseline_age + pct_weight_cha
                    'sl_libs' = c('SL.polymars', 'SL.lm')),

    ### E * mu0 * u/(1-u) ~ Lc, A = 0, R = 1, Y
    'chi' = list('type' = 'super_learner',
                 'Y' = 'Emu0_uratio',
                 'X' = c('race_source_value', 'gender_source_value', 'baseline_age', 'pct_weight_change
                 'formula' = ~race_source_value + gender_source_value + baseline_age + pct_weight_chang
                 'sl_libs' = c('SL.polymars', 'SL.lm'))
  )
```

Next, we fun the analysis

```
set.seed(3810)
results <-
  eif_estimator(df = df_analysis,
                models = model_list,
                n_splits = 2)
```

```
## Fitting ETA on Split [1/2]
## Fitting EPSILON on Split [1/2]
## Fitting MU0 on Split [1/2]
## Fitting U on Split [1/2]
## Fitting XI on Split [1/2]
## Fitting GAMMA on Split [1/2]
## Fitting CHI on Split [1/2]
## Fitting ETA on Split [2/2]
## Fitting EPSILON on Split [2/2]
## Fitting MU0 on Split [2/2]
## Fitting U on Split [2/2]
## Fitting XI on Split [2/2]
## Fitting GAMMA on Split [2/2]
## Fitting CHI on Split [2/2]
```

```
results$tau_hat
```

```
## # A tibble: 1 x 2
##   att_hat      sd
##     <dbl>   <dbl>
## 1 -0.0591 0.00486
```

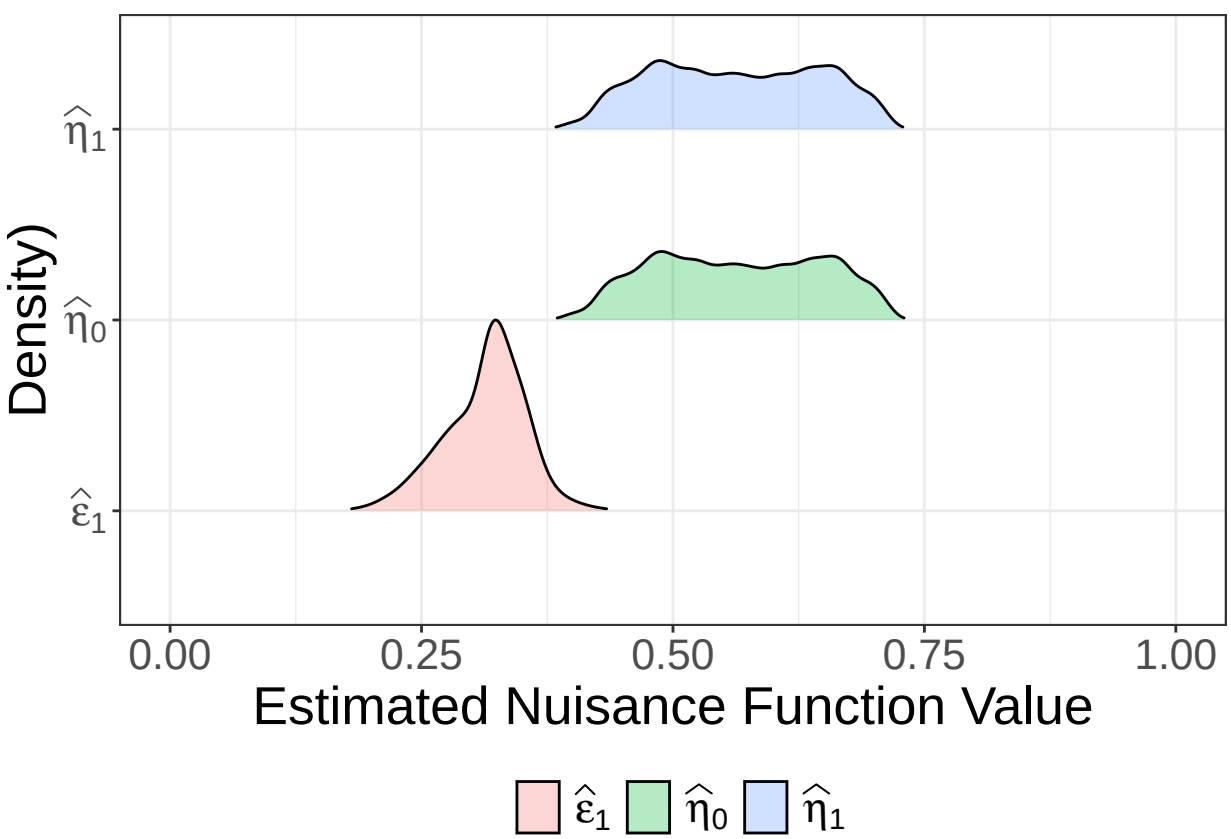
We see that $\widehat{\theta}_{\text{EIF}}$ = -0.0591006, recovering the true value of -0.06.

Diagnostics and Visualizations

To validate the stability of $\widehat{\theta}_{\text{EIF}}$, we produce some plots and additional checks, following Appendix S7.

```
### Elig-Related Nuisances
nuisance_elig <-
  results$df_EIF %>%
  pivot_longer(cols = c('eta1_hat', 'eta0_hat', 'epsilon1_hat'),
               names_to = 'nuisance_fx',
               values_to = 'value_hat')


ggplot(nuisance_elig, aes(x = value_hat, y = nuisance_fx)) +
  geom_density_ridges(aes(fill = nuisance_fx),
                      rel_min_height = 0.01,
                      scale = 1,
                      alpha = 0.3) +
  scale_fill_discrete(labels = c(expression(widehat(epsilon)[1]), expression(widehat(eta)[0]), expressi
  scale_x_continuous(limits = c(0,1)) +
  scale_y_discrete(labels = c(expression(widehat(epsilon)[1]), expression(widehat(eta)[0]), expression(
  labs(x = 'Estimated Nuisance Function Value',
       y = 'Density)',
       fill = '') +
  theme(axis.text = element_text(size = 16),
        strip.text = element_text(size = 14),
        legend.text = element_text(size = 16))
```



We notice that $\widehat{\eta}_0$ and $\widehat{\eta}_1$ are bounded away from 0 and have a high degree of overlap suggesting differential missingness is not an issue. Additionally, adjusted eligibility rate (given completely observed information),

$$\mathbb{P}_n[\widehat{\varepsilon}(\boldsymbol{L}^*, Y)] =$$

0.314. This is nearly identical to the observed rate $\frac{1822}{1822+4195} = 0.303$, another good calibration sanity check.

```
### Effective Sample Size Computation
results$df_EIF %>%
  filter(R == 1, eligible == 1) %>%
  summarise('n' = n(),
            'ess' = sum(1/eta1_hat)^2/sum(1/eta1_hat^2))
```

```
## # A tibble: 1 x 2
##       n   ess
##   <int> <dbl>
## 1  1822 1783.
```

Next we see that the Kish-like effective sample size contribution is nearly identical to the true number of ascertainably eligible patients, further confirming hat η is well calibrated.

Now, we turn our attention to the distribution of complete case propensity scores $\widehat{u}$. Note that these only contribute in eligible complete cases undergoing RYGB ($A = 0$).

```
### Distribution of u_hat
results$df_EIF %>%
  filter(A == 0, R == 1, eligible == 1) %>%
  summarise('min' = min(u_hat),
            'q01' = quantile(u_hat, 0.01),
            'q25' = quantile(u_hat, 0.25),
            'mean' = mean(u_hat),
            'median' = median(u_hat),
            'q75' = quantile(u_hat, 0.75),
            'q99' = quantile(u_hat, 0.99),
            'max' = max(u_hat))
```

```
## # A tibble: 1 x 8
##     min   q01   q25  mean median   q75   q99   max
##   <dbl> <dbl> <dbl> <dbl>  <dbl> <dbl> <dbl> <dbl>
## 1 0.386 0.480 0.602 0.647  0.649 0.699 0.785 0.804
```

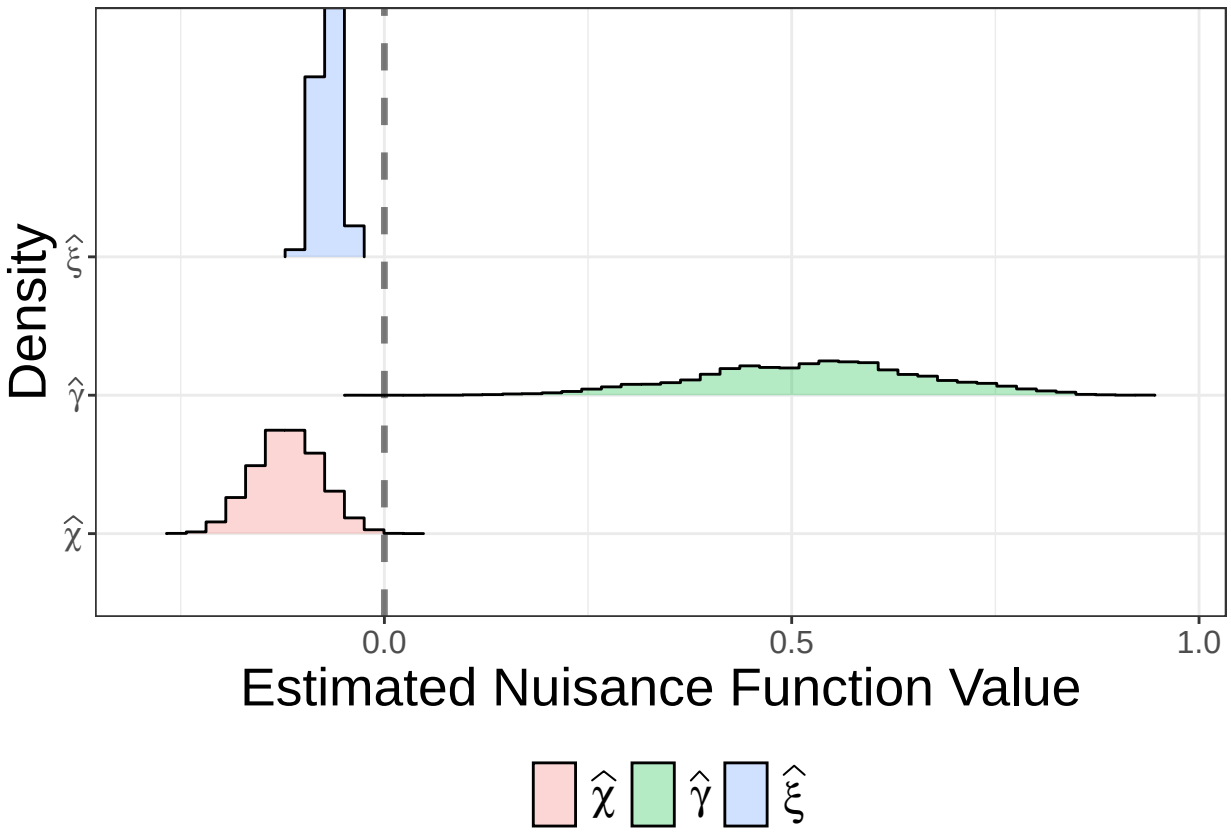
All values of $\widehat{u}$ range are clearly bounded away from 0 and 1, so we aren't worried about having to truncate extreme weights.

Finally, we come to the distribution of nested nuisances $(\widehat{\xi}, \widehat{\gamma}, \widehat{\chi})$.

```
nuisance_nest  <-
  results$df_EIF %>%
  pivot_longer(cols = c('xi_hat', 'gamma_hat', 'chi_hat'),
               names_to = 'nuisance_fx',
               values_to = 'value_hat')

ggplot(nuisance_nest, aes(x = value_hat, y = nuisance_fx)) +
  geom_vline(xintercept = 0, lty = 2, alpha = 0.5, lwd = 1.2) +
```

```
  stat_binline(aes(fill = nuisance_fx),
               draw_baseline = F,
               bins = 50,
               alpha = 0.3) +
  scale_y_discrete(labels = c(expression(widehat(chi)), expression(widehat(gamma)), expression(widehat(
  scale_fill_discrete(labels = c(expression(widehat(chi)), expression(widehat(gamma)), expression(wideha
  labs(x = 'Estimated Nuisance Function Value',
       y = 'Density',
       fill = '') +
  theme(axis.text = element_text(size = 12),
        strip.text = element_text(size = 14),
        legend.text = element_text(size = 16))
```



There's not a huge degree of concentration at 0 or skew to any of these distributions, further confirming the behavior of underlying nuisances u and μ_0. Altogether, there's nothing overly suggestive of instability. We report the final value of $\widehat{\theta}_{\text{EIF}}$ as above

```
results$tau_hat
```

```
## # A tibble: 1 x 2
##   att_hat      sd
##     <dbl>   <dbl>
## 1 -0.0591 0.00486
```